

Nios[®] V/c Processor PIO and OCM Design

Agilex[™] 7 FPGA

Contents

1. Theory of Operation	3
1.1. IP Cores	3
2. Directory Structure	4
3. Quick Start Guide	4
4. Generating the Design	5
4.1. Building Hardware Design	5
4.2. Building Software Design	5
6. Programming the Design	6
7. Expected Results	7

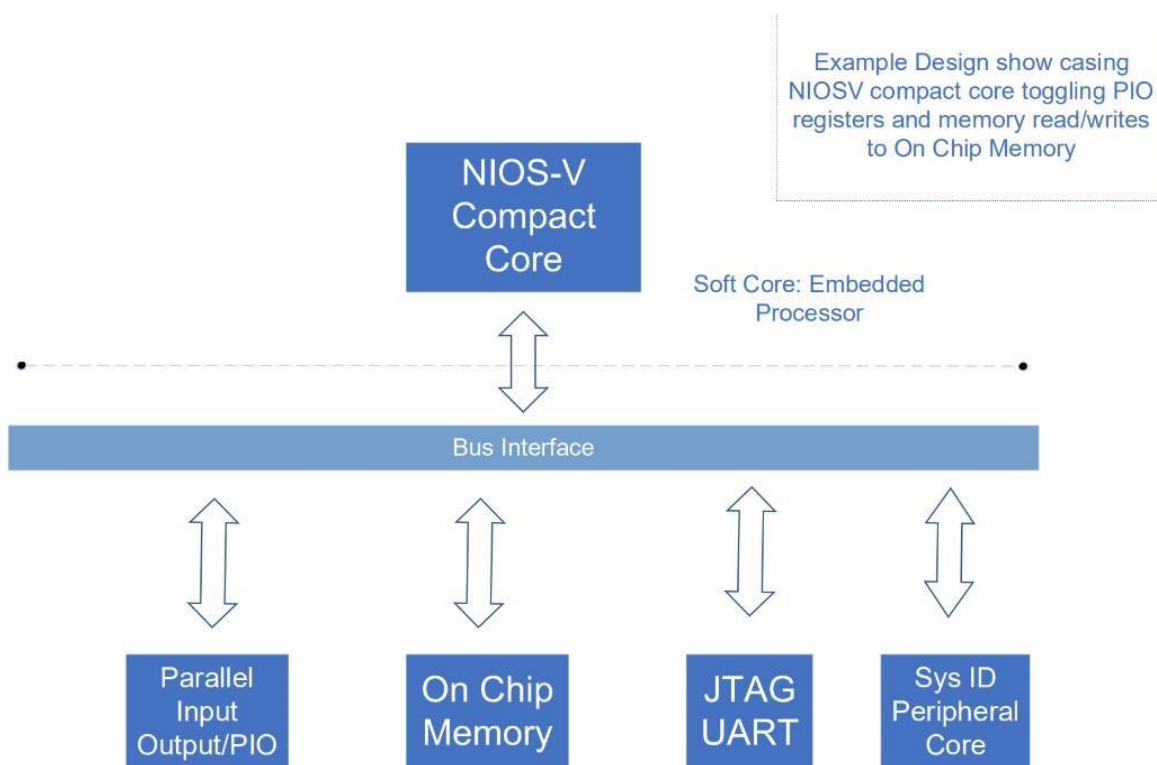
1. Theory of Operation

This example design includes a Nios V/c compact embedded processor connected to the Parallel Input Output – PIO IP, On Chip RAM JTAG UART IP and System ID Peripheral core IP

The objective of the design is to accomplish:

1. Toggle the PIO registers which in turn toggles the LED connected on dev kit and readback the registers
2. Write/Read to and from the On Chip memory connected to the embedded processor
3. Readback and print the System Peripheral core register to check of the value written in the IP during configuration

Figure 1-1. Block Diagram



1.1. IP Cores

The following IPs are used in this design.

- Nios V/c soft processor core
- On Chip RAM-II
- JTAG UART
- PIO IP
- System ID

2. Directory Structure

The directory structure is explained below:

Root directory: agf014ea-dev-devkit/niosv_c/pio_ocm

Directories	Content
docs	Document capturing the details of the example design
img	Block diagram for the example designs
ready_to_test	Prebuilt binaries (FPGA .sof file and Application .elf file) which can be programmed on the board and tested
sources	Files needed to create the design
- hw	- Necessary hardware files (.sv, .v, .ip) of the design
- scripts	- This folder consists of scripts to build the design
- sw/app	- This folder contains software application files

3. Quick Start Guide

You may try the example on the target development kit using the prebuilt binaries. The respective FPGA sof and Application elf files are found in ready_to_test folder.

Refer to [Chapter 5 Programming the Design](#) for the steps to program these files.

4. Generating the Design

The implementation of Nios® V processor system requires a hardware design developed using the Quartus® Prime software. After that, it is required to develop the software design that complements the processor system.

4.1. Building Hardware Design

Begin designing the system hardware by instantiating the Nios® V processor and its peripherals into Platform Designer. After configuring the system assignments and constraints, complete the hardware design by performing a successful compilation.

We will be using `sources/scripts/build_sof.py` to develop the hardware design.

This script will create a new platform design, add the IPs into the design, makes the connections, compiles the design and generates the `.sof` file.

Steps to execute the script are as follows:

1. Invoke the Nios V Command Shell in the terminal
2. Run the following command in the terminal from top level project directory:
`> quartus_py ./scripts/build_sof.py`
3. The Quartus tool will compile the design and generate the output files into `<Root Directory>/sources/hw/output_files`

4.2. Building Software Design

After the processor system is ready, you may begin building the software design. It consists of the following steps:

1. Create a board support package (BSP) project.
2. Create a Nios® V processor application project with source code.
3. Build the application.
4. Convert the application `.elf` file into `.hex` file for memory initialization.

To create software application, run the following commands in the terminal:

```
> niosv-bsp -c --quartus-project=hw/top.qpf --
qsys=hw/qsys_top.qsys --type=hal --script=sw/bsp-update-small-
driver.tcl sw/bsp/settings.bsp

> niosv-app --bsp-dir=sw/bsp --app-dir=sw/app --srcs=sw/app/main.c

> cmake -S ./sw/app -B sw/app/build

> make -C sw/app/build

> elf2hex sw/app/build/app.elf -b 0x0 -w 32 -e 0xffffffff
hw/onchip_mem.hex
```

Note: It is recommended to clean the `app/build` project before regenerating `elf` (cmake and make).

5. Simulating the Design

You may carry on simulation for this design to evaluate its intended functions. During simulation, the action of downloading .elf file is unable to be simulated. Thus, the processor memory is instead initialized with the application .hex file.

Use the following commands to run the simulation:

```
# Generate Testbench from Platform Designer.

# Generate -> Generate Testbench System

> cp ./hw/onchip_mem.hex ./qsys_top_tb/qsys_top_tb/sim/mentor

> cd hw/qsys_top_tb/qsys_top_tb/sim/mentor/

> vsim &

> source msim_setup.tcl

> ld_debug

> run -all
```

6. Programming the Design

Use the Quartus® Prime Programmer tool and the Ashling* RiscFree* IDE to program the Nios® V processor-based system into the FPGA and to run your application.

Program the generated sof, then download the elf file on the board according to the command below.

```
> quartus_pgm --cable=1 -m jtag -o 'p;ready_to_test/top.sof'

#----- Download the elf file on the board ----#

> niosv-download -g ready_to_test/app.elf -c 1

#----- Verify the output on the terminal by using the following command in the
terminal ----#

> juart-terminal -d 1 -c 1 -i 0
```

Note: Reduce the JTAG clock frequency to 6MHz before programming the application .elf file on the board using the command: `jtagconfig --setparam 1 JtagClock 6M`

7. Expected Results

The following is the output as observed on the JTAG UART terminal. The output is analogous to the logic from the application code. Users should be able to observe the same output on their terminal/setup.

Figure 1-2. Expected Result

```
Hello World
Application will execute Memory Test & PIO toggle test
Starting Memory test
value at memory location 0x0 with offset 0 is 0x2a00006f
After write: value at memory location 0x0 with offset 0 is 0xa5a5a5a5
Memory write test PASSED
value at memory location 0x0 with offset 4 is 0x0
After write: value at memory location 0x0 with offset 4 is 0xa5a5a5a5
Memory write test PASSED
value at memory location 0x0 with offset 8 is 0x0
After write: value at memory location 0x0 with offset 8 is 0xa5a5a5a5
Memory write test PASSED
value at memory location 0x0 with offset 12 is 0x0
After write: value at memory location 0x0 with offset 12 is 0xa5a5a5a5
Memory write test PASSED
value at memory location 0x0 with offset 16 is 0x0
After write: value at memory location 0x0 with offset 16 is 0xa5a5a5a5
Memory write test PASSED
value at memory location 0x0 with offset 20 is 0x0
After write: value at memory location 0x0 with offset 20 is 0xa5a5a5a5
Memory write test PASSED
value at memory location 0x0 with offset 24 is 0x0
After write: value at memory location 0x0 with offset 24 is 0xa5a5a5a5
Memory write test PASSED
value at memory location 0x0 with offset 28 is 0x0
After write: value at memory location 0x0 with offset 28 is 0xa5a5a5a5
Memory write test PASSED
Memory Test Complete
```

```
PIO Test Start: Toggle PIO registers
Application to toggle the PIOs- [4:0]
DATA READBACK FROM PIO_0_BASE is 0x0
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x1
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x2
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x3
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x4
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x5
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x6
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x7
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x8
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x9
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0xa
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0xb
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0xc
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0xd
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0xe
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0xf
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x0
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x1
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x2
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x3
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x4
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x5
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x6
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x7
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x8
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x9
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0xa
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0xb
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0xc
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0xd
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0xe
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0xf
DATA MATCHED - TEST PASSED
```



```
DATA READBACK FROM PIO_0_BASE is 0x0
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x1
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x2
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x3
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x4
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x5
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x6
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x7
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x8
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x9
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0xa
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0xb
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0xc
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0xd
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0xe
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0xf
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x0
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x1
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x2
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x3
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x4
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x5
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x6
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x7
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x8
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0x9
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0xa
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0xb
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0xc
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0xd
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0xe
DATA MATCHED - TEST PASSED
DATA READBACK FROM PIO_0_BASE is 0xf
DATA MATCHED - TEST PASSED
NIOSV-PIO Test PASSED
Readback and check System ID Peripheral Core value
SYS ID matched with hardware with value 0xA5
```